CMPUT 461/561 Fall 2026

Lec 05 -- Parsing

© Denilson Barbosa



Outline

1. Types of parsing (syntactical, dependency, statistical?)
2. CYK parsing
3. Chomsky Normal Form (CNF)
    - Conversion CFG → CNF
4. Ambiguity and PCFGs



Types of parsing

- Syntactic parsing
- Statistical parsing
- Dependency parsing

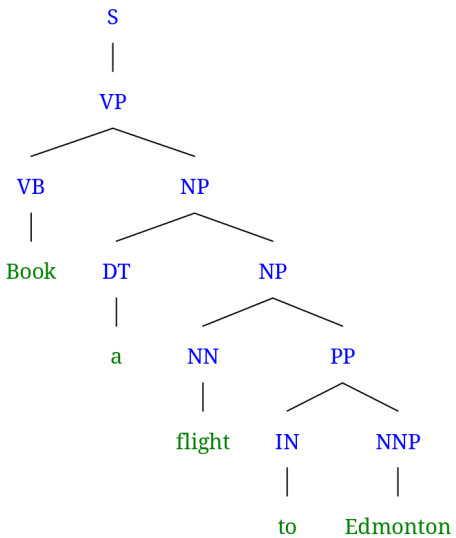
Syntactic parsing

Assigning a parse tree to a syntactically valid setence.

Input: sentence.
Output: parse tree (only) if the sentence is grammatical.

Parse tree:

- Leaf nodes are words.
- Inner nodes correspond to productions in the grammar.



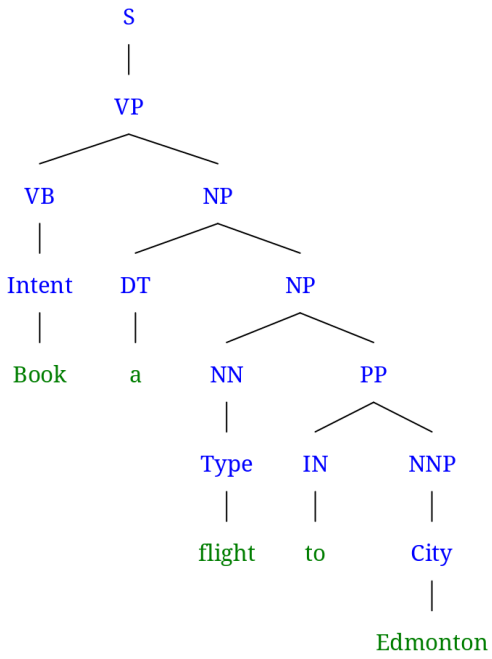
How do parse trees help?

Text interpretation:

- Identify constituents.
- Identify the "core" meaning of the sentence (ex: nouns and verbs).
- Answer "*who did what to whom?*" questions.

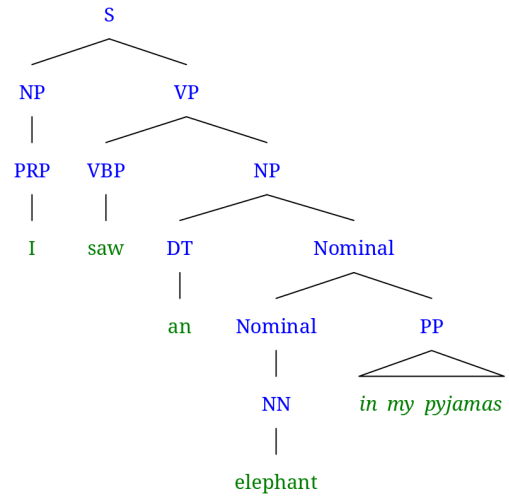
Application-specific tasks:

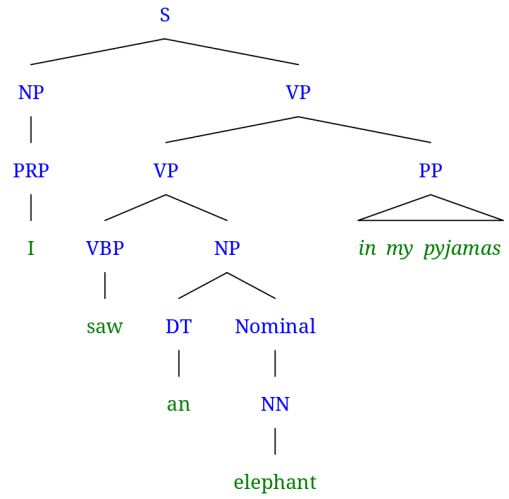
- We can **customize** the grammar for specific applications (ex: intent classification).

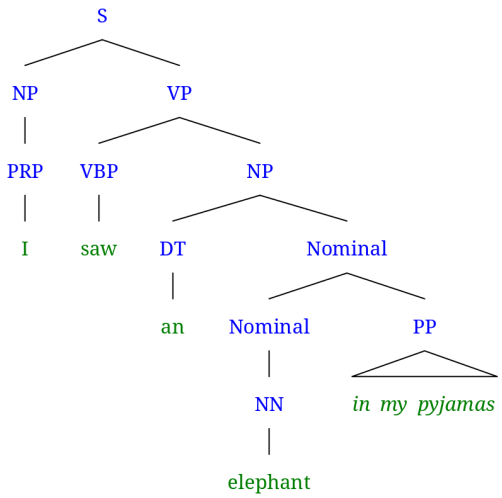


Ambiguous parses

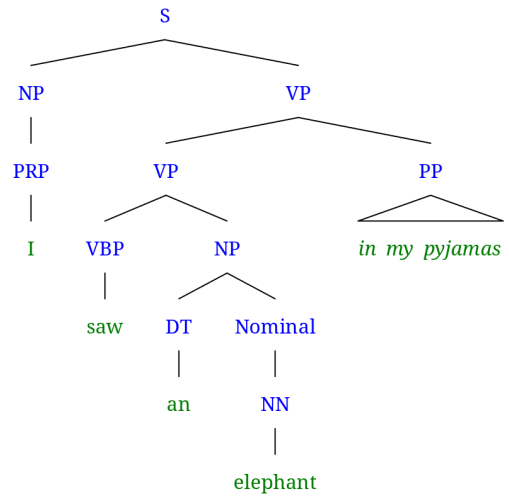
Some sentences can be parsed in multiple ways, each with a different meaning.







The elephant was in "my pyjamas".



I was in "my pyjamas" when **I** saw the elephant.

Probabilistic CFGs and parsing

A probabilistic CFG (PCFG) has probabilities assigned to the productions.

Probabilities **must sum up to 1.0** (for each non-terminal).

- Ex: two rules for VP in the grammar.

Probabilities are learned from parse trees from real sentences (ideally, parsed by humans).

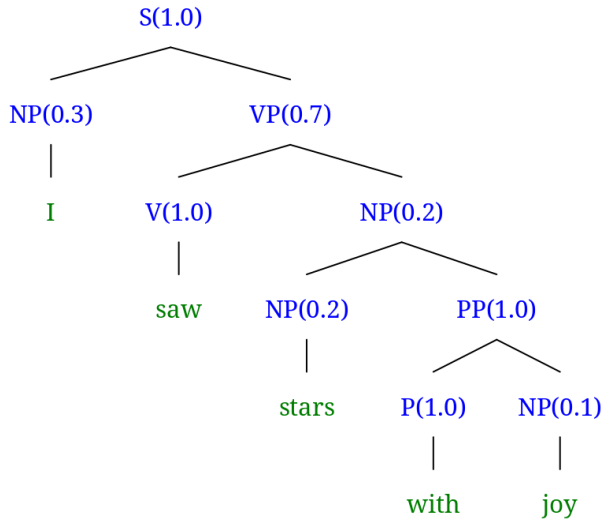
- Count how many times each rule is applied...

Example:

S $(1.0) \rightarrow$ NP VP	NP $(0.2) \rightarrow$ NP PP
VP $(0.7) \rightarrow$ V NP	NP $(0.3) \rightarrow$ I
VP $(0.3) \rightarrow$ VP PP	NP $(0.1) \rightarrow$ joy
PP $(1.0) \rightarrow$ P NP	NP $(0.2) \rightarrow$ stars
P $(1.0) \rightarrow$ with	NP $(0.1) \rightarrow$ saw
V $(1.0) \rightarrow$ saw	NP $(0.1) \rightarrow$ moons

According to this grammar, a verb followed by a noun phrase (`V NP`) is much more likely than a verb followed by a prepositional phrase (`V PP`).

Parse tree t_1

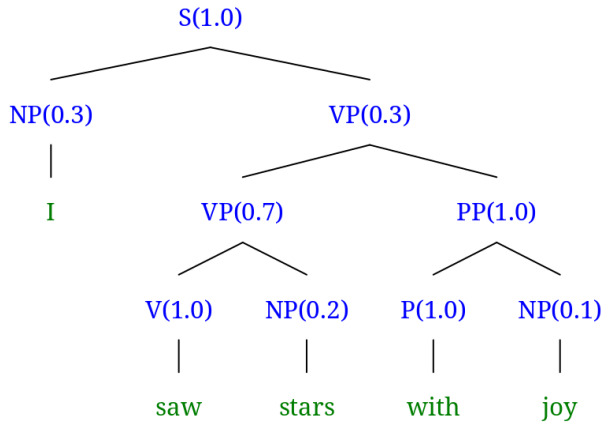


S (1.0) → NP VP
VP (0.7) → V NP
VP (0.3) → VP PP
PP (1.0) → P NP
P (1.0) → with
V (1.0) → saw

NP (0.2) → NP PP
NP (0.3) → I
NP (0.1) → joy
NP (0.2) → stars
NP (0.1) → saw
NP (0.1) → moons

What did **I** see? "stars with joy".

Parse tree t_2



S (1.0) → NP VP
VP (0.7) → V NP
VP (0.3) → VP PP
PP (1.0) → P NP
P (1.0) → with
V (1.0) → saw

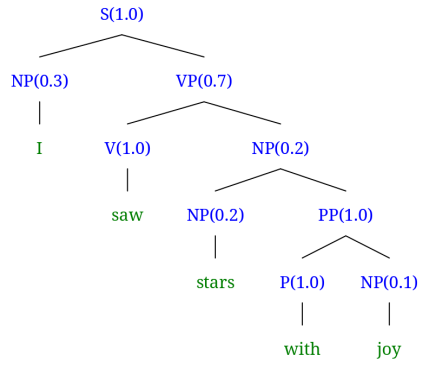
NP (0.2) → NP PP
NP (0.3) → I
NP (0.1) → joy
NP (0.2) → stars
NP (0.1) → saw
NP (0.1) → moons

What did **I** see? "stars".

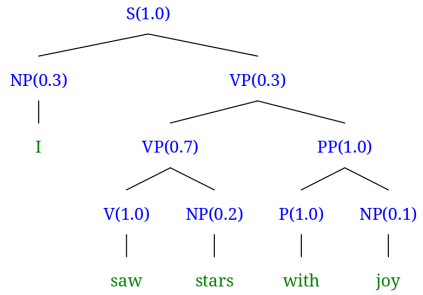
How did I see them? "with joy".

Disambiguating sentences with a PCFG

The probability of a parse (tree) is the **product** of the probabilities of all productions.



$$P(t_1) = 1 \cdot 0.3 \cdot 0.7 \cdot 1 \cdot 0.2 \cdot 0.2 \cdot 1 \cdot 1 \cdot 0.1$$
$$P(t_1) = 0.00084$$



$$P(t_2) = 1 \cdot 0.3 \cdot 0.3 \cdot 0.7 \cdot 1 \cdot 1 \cdot 0.2 \cdot 1 \cdot 0.1$$
$$P(t_2) = 0.00126$$

↠ t_2 is the **most likely parse!**



ASIDE: two levels of ambiguity

Lexical (word-level)

Happens when a single word may have more than one meaning.

- Ex: `stars` can be a verb or a noun.

Syntactical (sentence-level)

Happens when a sentence can be parsed in more than one way.

- Ex: `I saw stars with joy`

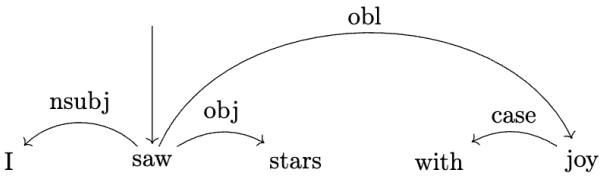
Ambiguity is solved when we **choose** a parse tree for the sentence.

Probabilistic grammars capture different possible grammatical structures with their likelihood, thus helping us disambiguate text.

Dependency parsing

Dependency grammars focus on the relations between words, rather than between constituents.

Vertices in dependency trees represent words, while the arcs capture the relationships between words.

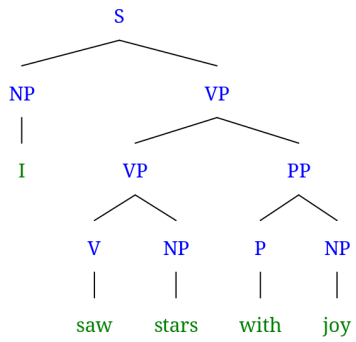


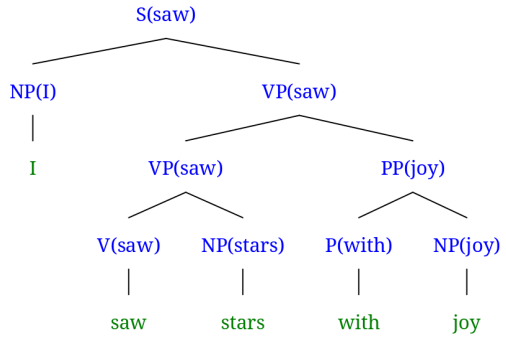
Example dependencies:

- **nsubj**: `I` is the subject of `saw` (who saw stars? "**I**")
- **obj**: `stars` is the object of `saw` (what did I see? "**stars**")
- **obl** (oblique nominal): `joy` is an adverb (how did I see stars? "**with joy**")
- **case**: `with` is a preposition of `joy`

Building dependency trees: step 1

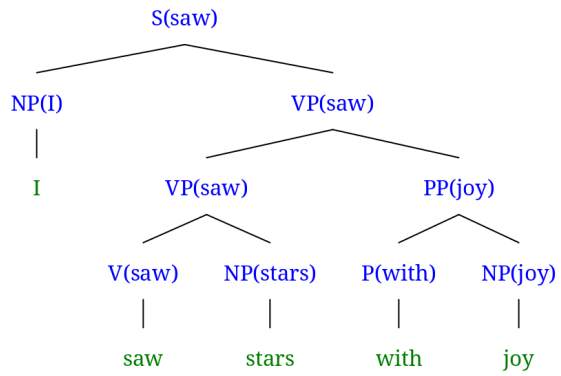
Starting from a constituency parse of the sentence, identify the **head words** of each constituent:

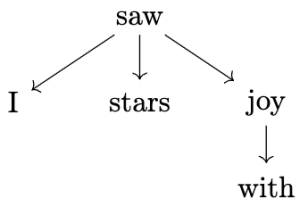




The head word of a constituent is its **most important** word (usually a verb or noun).

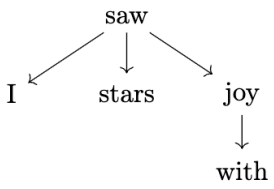
Head words and dependencies

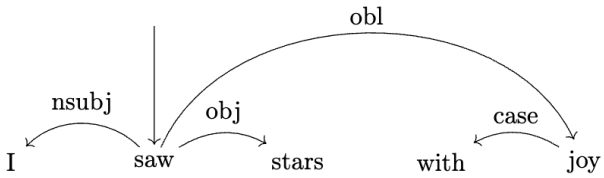




Verbs take precedence over nouns; therefore **saw** is the head word of the entire sentence.

Building dependency trees: step 2





Classify each edge in the tree, based on the words.

- Several annotated datasets exist for this task.

The head word of the sentence (**saw**) has no parent; this is normally indicated by an unlabeled edge coming onto it. (Sometimes, that edge is labelled "root".)

Universal dependency relations

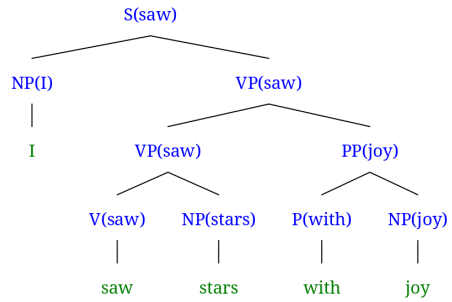
https://universaldependencies.org/u/dep/index.html

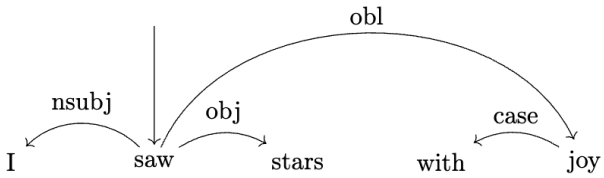
	Nominals	Clauses	Modifier words	Function Words
Core arguments	nsubj obj iobj	csubj ccomp xcomp		
Non-core dependents	obl vocative expl dislocated	advcl	advmod* discourse	aux cop mark
Nominal dependents	nmod appos nummod	acl	amod	det clf case
Coordination	**Headless**	**Loose**	**Special**	**Other**
conj cc	fixed flat	list parataxis	compound orphan goeswith reparandum	punct root dep

Some examples of dependency relations

relation	meaning	example
nsubj	subject	She gave me a raise → `nsubj(gave, she)`
obj	direct object	She gave me a raise → `obj(gave, raise)`
iobj	indirect object	She gave me a raise → `iobj(gave, me)`
nmod	nominal modifier	The office of the Chair → `nmod(office, Chair)`
nummod	numerical modifier	I ate three peas → `nummod(peas, three)`
amod	adjectival modifier	I love warm bread → `amod(bread, warm)`
conj	conjunct	Mom is fair and nice → `conj(fair, nice)`
vocative	vocative	Guys, wait for me → `vocative(wait, Guys)`

Dependency vs constituency parsing





Dependency trees have **fewer** and **more informative** edges.

- It is easier to build Natural Language Understanding tools with dependencies.
- But dependency parsers are probabilistic (trained on annotated data).



Cocke-Younger-Kasami (CYK) parsing

Building constituency trees.

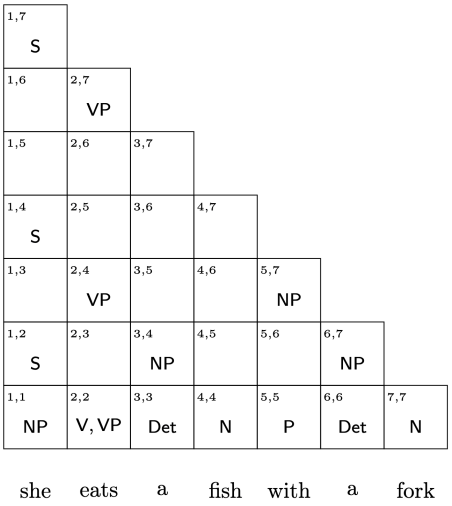
https://en.wikipedia.org/wiki/CYK_algorithm

CYK parsing

The Cocke–Younger–Kasami (CYK) parser is a recursive, bottom-up parser that uses **dynamic programming** to avoid parsing the same constituent more than once.

It fills half of a matrix (the "memoization" table) of size $n \times n$, where n is the length of the sentence.

It **recognizes** the sentence as grammatical if the start symbol is at cell $(1, n)$.



https://en.wikipedia.org/wiki/Dynamic_programming

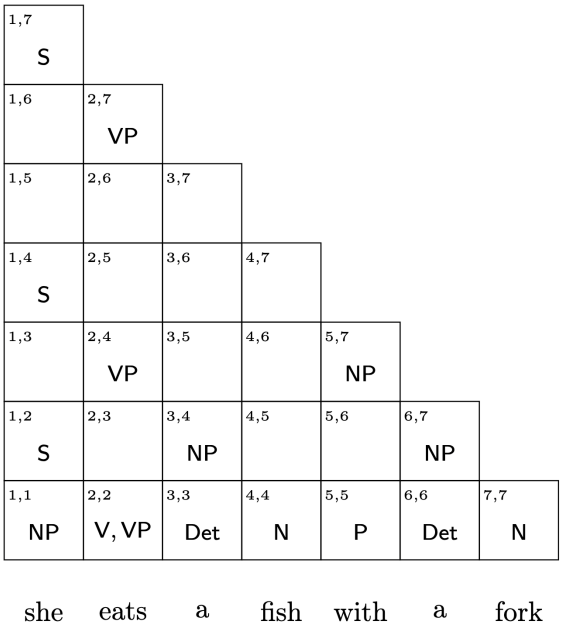
Each cell in the matrix covers a **phrase** (i.e., a range of words). Examples:

- Cell (1,1) → `she`
- Cell (2,4) → `eats` `a` `fish`
- Cell (3,5) → `a` `fish` `with`

Each cell contains all productions that generate the phrase covered by the cell.

The sentence in the example is **grammatical**, because the start symbol (S) is in cell (1,7).

- Recall that cell (1,7) "covers" the entire sentence.

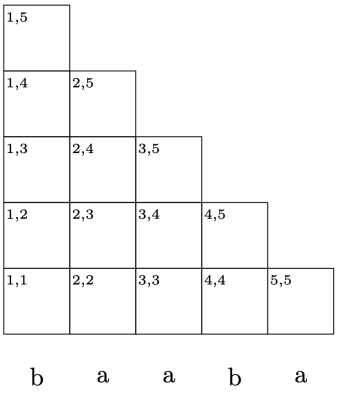


Each cell in the matrix covers a range of words. Examples:

- Cell (1,1) → `b`
- Cell (2,4) → `a` `a` `b`
- Cell (3,5) → `a` `b` `a`

Starting with an empty table, we **fill each row** (bottom-up).

We put in each cell **all productions** corresponding to the range of words in the cell.



$S \to A\,B$
$S \to B\,C$
$A \to B\,A$
$A \to \mathrm{a}$
$B \to C\,C$
$B \to \mathrm{b}$
$C \to A\,B$
$C \to \mathrm{a}$

Productions that apply for **row 1**:

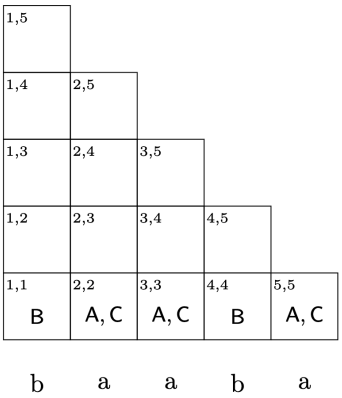
$\mathsf{B} \to \mathrm{b}$ $\mathsf{A} \to \mathrm{a}$ $\mathsf{C} \to \mathrm{a}$

Filling out cell (1,2), covering phrase `b` `a`.

Instead of looking at the terminals (again), we consider **all combinations of non-terminals** in cells (1,1) and (2,2):

- B A is generated by A → B A
- B C is generated by S → B C

Thus, we add both A and S to (1,2).

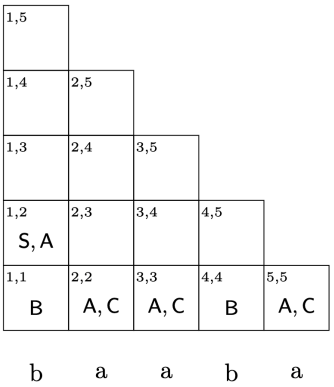


$S \to A\,B$
$S \to B\,C$
$A \to B\,A$
$A \to \mathrm{a}$
$B \to C\,C$
$B \to \mathrm{b}$
$C \to A\,B$
$C \to \mathrm{a}$

Filling out all cells in the second row (bottom-up).

Note that all cells look like $(i, i+1)$.

Look for productions that generate combinations X Y of non-terminals where X is in (i, i) and Y is in $(i+1, i+1)$.



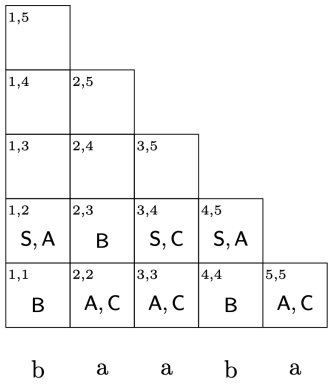
$S \to A\ B$
$S \to B\ C$
$A \to B\ A$
$A \to \mathrm{a}$
$B \to C\ C$
$B \to \mathrm{b}$
$C \to A\ B$
$C \to \mathrm{a}$

Filling out cell (1,3) -- phrase `b` `a` `b`.

There are two combinations of cells that, together, cover the phrase:

- Cells (1,2) and (3,3)
- Cells (1,1) and (2,3)

For each combination of cells, look for productions that generate combinations X Y of non-terminals where X is in the first cell and Y is in the second.



$S \to A\ B$
$S \to B\ C$
$A \to B\ A$
$A \to \mathrm{a}$
$B \to C\ C$
$B \to \mathrm{b}$
$C \to A\ B$
$C \to \mathrm{a}$

- (1,2) + (3,3): S A; S C; A A; A C
- (1,1) + (2,3): B B

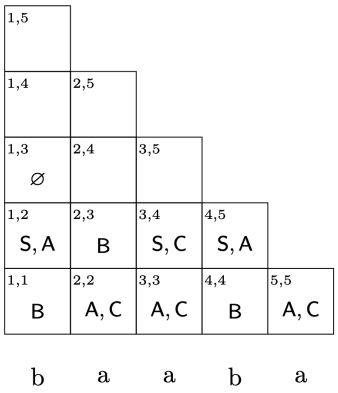
No production generates any of those combinations!

Filling out cell (2,4) -- phrase `a` `a` `b`.

- (2,3) + (4,4) = ?
- (2,2) + (3,4) = ?

Filling out cell (3,5) -- phrase `a` `b` `a`.

- (3,4) + (5,5) = ?
- (3,3) + (4,5) = ?



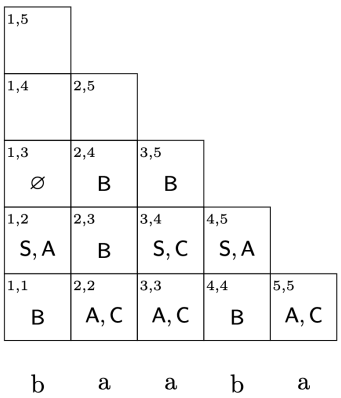
$S \to A\,B$
$S \to B\,C$
$A \to B\,A$
$A \to \mathrm{a}$
$B \to C\,C$
$B \to \mathrm{b}$
$C \to A\,B$
$C \to \mathrm{a}$

Filling out cell (1,4) requires even more combinations:

- (1,3) + (4,4) = ?
- (1,2) + (3,4) = ?
- (1,1) + (2,4) = ?

Filling out cell (2,5):

- (2,4) + (5,5) = ?
- (2,3) + (4,5) = ?
- (2,2) + (3,5) = ?

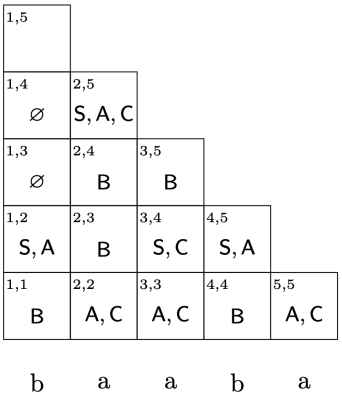


$S \to A\,B$
$S \to B\,C$
$A \to B\,A$
$A \to \mathrm{a}$
$B \to C\,C$
$B \to \mathrm{b}$
$C \to A\,B$
$C \to \mathrm{a}$

Note that, even though there are more combinations, all cells used in each of them have already been computed!

Filling out cell (1,5) requires even more combinations:

- (1,4) + (5,5) = ?
- (1,3) + (4,5) = ?
- (1,2) + (3,5) = ?
- (1,1) + (2,5) = ?

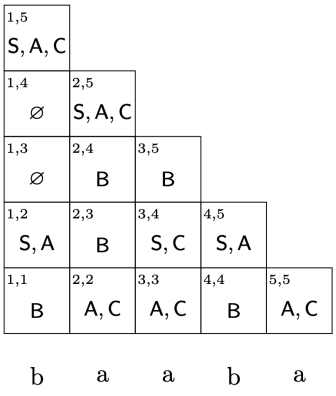


$S \to A\,B$
$S \to B\,C$
$A \to B\,A$
$A \to \mathrm{a}$
$B \to C\,C$
$B \to \mathrm{b}$
$C \to A\,B$
$C \to \mathrm{a}$

Note that, even though there are more combinations, all cells used in each of them have already been computed!

At the end, cell (1,5) contains S, so **the sentence is grammatical**!

Note that **no cell is computed more than once** (that's dynamic programming!)



$S \to A\,B$
$S \to B\,C$
$A \to B\,A$
$A \to \mathrm{a}$
$B \to C\,C$
$B \to \mathrm{b}$
$C \to A\,B$
$C \to \mathrm{a}$

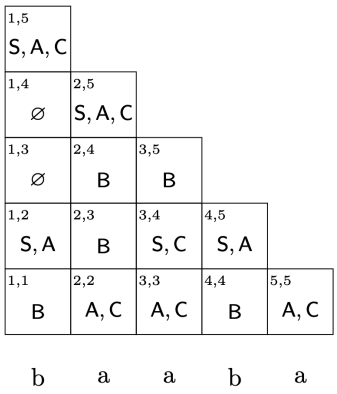
ASIDE: Complexity analysis

Assuming the grammar is fixed (so its size is a constant in the analysis).

Input: phrase with n words.

Fill out (half of) a $n \times n$ table = $O(n^2)$ cells!

In each cell, look at $O(n)$ combinations of **two=$O(1)$ previously filled out** cells.



$S \to A\,B$
$S \to B\,C$
$A \to B\,A$
$A \to \mathrm{a}$
$B \to C\,C$
$B \to \mathrm{b}$
$C \to A\,B$
$C \to \mathrm{a}$

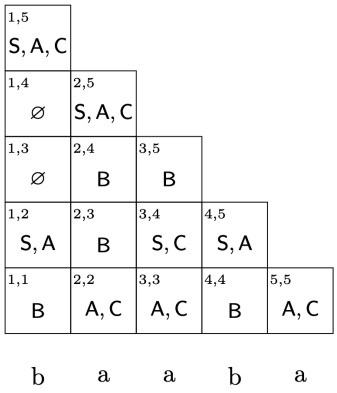
Therefore, **cost is $O(n^3)$**.

ASIDE: Complexity analysis

At each cell, look at $O(n)$ combinations of **two=$O(1)$ previously filled out** cells.

Q: Why only two cells?

A: because there are **at most** two symbols at the RHS of every production rule!



$S \to A\,B$
$S \to B\,C$
$A \to B\,A$
$A \to \mathrm{a}$
$B \to C\,C$
$B \to \mathrm{b}$
$C \to A\,B$
$C \to \mathrm{a}$

The grammar is in **Chomsky Normal Form (CNF)**.



Chomsky Normal Form



REPEAT: Formal definition of a CFG

A Context-Free Grammar (CFG) is a tuple (N, Σ, R, σ) where:

- N is a set of **non-terminal** symbols (also called **variables**).
- Σ is a set of **terminal symbols** (disjoint from N and sometimes called the *lexicon*).
- R is a set of **rules** (also called **productions**), each of the form $A \to \beta$,
  where A is a non-terminal, and
  β is a string of symbols from (the infinite set) $(\Sigma \cup N)*$
- $\sigma \in N$ is a designated **start symbol** (normally **S**).

Chomsky Normal Form (CNF)

Restricts the form of the production rules (LHS → RHS) as follows:

The right-hand side (RHS) of a rule can only have **one or two elements**:

- A single terminal.
- Two non-terminals.

ϵ can only appear in the RHS of the start symbol (S).

That is, a grmmar is in CNF if and only if all of its rules have the form:

- $X \to Y\ Z$
- $X \to \mathrm{x}$
- $S \to \epsilon$

Converting a CFG into CNF

Every CFG can be converted into a weakly equivalent CNF grammar.
https://en.wikipedia.org/wiki/Chomsky_normal_form

The algorithm performs a series of rewriting steps.

START: if the start symbol appears in the RHS of some rule, introduce a new start symbol S_0 and add rule

$S_0 \to S$

TERM: rewrite $A \to X_1 \cdots \mathrm{a} \cdots X_n$ into

$N_a \to \mathrm{a}$
$A \to X_1 \cdots N_a \cdots X_n$

BIN: rewrite $A \to X_1\, X_2\, X_3 \cdots X_n$ into

$A \to X_1\, A_1$

$A_1 \to X_2\, A_2$

$\ldots$

$A_{n-2} \to X_{n-1}\, X_n$

UNIT: if $B \to X_1\, X_2 \cdots X_n$ and $A \to B$ is in the grammar, replace it with

$A \to X_1\, X_2 \cdots X_n$

DEL: eliminate rules $A \to \epsilon$ where A is not the start symbol.

Notation: if $A \to \epsilon$ then A is *nullable*; if $A \to X_1 \cdots X_n$ exists and every X_i is *nullable*, then A is also *nullable*.

If A is nullable, replace $A \to X_1 \cdots X_n$ by all versions of the rule where some *nullable* X_i is omitted.

Example

CFG:

$S \to A\, B\, C$

$A \to a\, C \mid \epsilon \mid D$

$B \to b\, B \mid \epsilon \mid A$

$C \to A\, c \mid \epsilon \mid C\, c$

$D \to a\, a$

STEP 1 (DEL): dealing with *nullables*.

S, A, B, C are all *nullable*.

Versions of rules with some *nullable* omitted:

$S \to A\, B \mid A\, C \mid B\, C$

$A \to a$

$B \to b$

$C \to c$

Those need to be **added to the CFG**, allowing us to remove all $X \to \epsilon$ productions.

We also need to add $S \to \epsilon$.

Example

CFG after STEP 1:

$S \to ABC \mid AB \mid AC \mid BC \mid \epsilon$

$A \to aC \mid D \mid a$

$B \to bB \mid A \mid b$

$C \to Ac \mid Cc \mid c$

$D \to aa$

STEP 2 (UNIT):

Remove $A \to D$ and $B \to A$

and

add rules

- $A \to aa$
- $B \to aC \mid aa \mid a$

Example

CFG after STEP 2:

$S \to ABC \mid AB \mid AC \mid BC \mid \epsilon$

$A \to aC \mid aa \mid a$

$B \to bB \mid aC \mid aa \mid a \mid b$

$C \to Ac \mid Cc \mid c$

$D \to aa$

STEP 3 (TERM):

Add rules

- $N_a \to a$
- $N_b \to b$
- $N_c \to c$

and replace all occurrences of terminals in the RHS (except $X \to x$).

- Ex: aC becomes $N_a C$.

Example

CFG after STEP 3:

$S \to A\,B\,C \mid A\,B \mid A\,C \mid B\,C \mid \epsilon$

$A \to N_a\,C \mid N_a\,N_a \mid a$

$B \to N_b\,B \mid N_a\,C \mid N_a\,N_a \mid a \mid b$

$C \to A\,N_c \mid C\,N_c \mid c$

$D \to N_a\,N_a$

$N_a \to a$

$N_b \to b$

$N_c \to c$

STEP 4 (BIN):

Replace $S \to A\,B\,C$ with

$S \to A\,M$

$M \to B\,C$

Example

CFG after STEP 4:

$S \to A\,M \mid A\,B \mid A\,C \mid B\,C \mid \epsilon$

$M \to B\,C$

$A \to N_a\,C \mid N_a\,N_a \mid a$

$B \to N_b\,B \mid N_a\,C \mid N_a\,N_a \mid a \mid b$

$C \to A\,N_c \mid C\,N_c \mid c$

$D \to N_a\,N_a$

$N_a \to a$

$N_b \to b$

$N_c \to c$

We don't have any S in any RHS, so there's no need to use the **START** transformation.

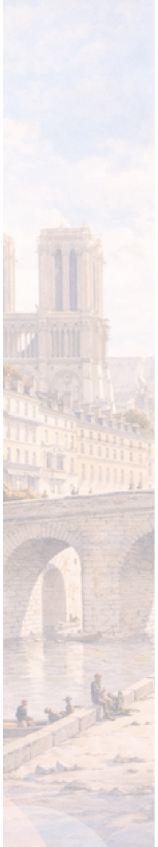
The conversion into CNF is complete.

Check that all rules have the form:

- $X \to Y\,Z$
- $X \to x$
- $S \to \epsilon$



Ambiguity and PCFGs



REPEAT: Ambiguity

Lexical (word-level)

Happens when a single word may have more than one meaning.

- Ex: `saw` can be a verb or a noun.

Syntactical (sentence-level)

Happens when a sentence can be parsed in more than one way.

- Ex: `I saw stars with joy`

Both forms of ambiguity are solved when we **decide** how to parse the sentence.

Probabilistic grammars capture different possible grammatical structures with their likelihood, thus whelping with disambiguating text.

Ambiguity and CYK

CYK computes **all possible parses** of a sentence.

If multiple parses are allowed, both will appear in the last cell.

Cell (2,5) has:

- VP_1 because of (2,2)+(3,5)
- VP_2 because of (2,3)+(4,5)

1,5 S_1, S_2				
1,4	2,5 VP_1, VP_2			
1,3 S_1	2,4	3,5 NP		
1,2	2,3 VP_1	3,4	4,5 PP	
1,1 NP	2,2 V, NP	3,3 NP	4,4 P	5,5 NP
I	saw	stars	with	joy

$S_1 \rightarrow NP\ VP_1$
$S_2 \rightarrow NP\ VP_2$
$VP_1 \rightarrow V\ NP$
$VP_2 \rightarrow VP_1\ PP$
$VP_2 \rightarrow VP_2\ PP$
$PP \rightarrow P\ NP$
$P \rightarrow$ with
$V \rightarrow$ saw
$NP \rightarrow NP\ PP$
$NP \rightarrow$ I | joy | saw
$NP \rightarrow$ stars | moons

Replaced VP with two non-terminals, one for each form of phrase.



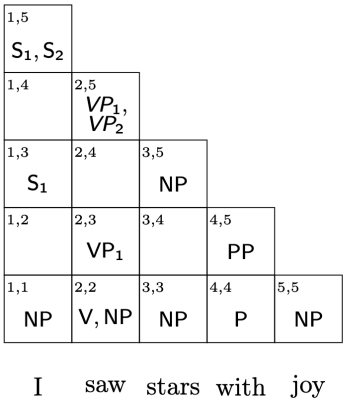
Final comments on CYK

Building the parse tree

We can build the tree by **traversing the matrix backwards** from the final cell.

This can be easily done by keeping **pointers** for each non-terminal in each cell.

NOTE: the CNF tree might not be as useful/readable as the tree that would be produced with the original CFG.



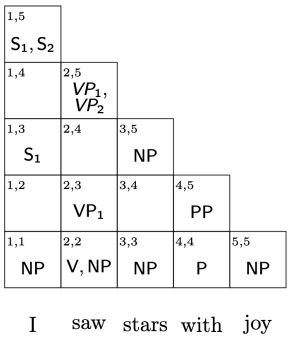
$S_1 \rightarrow NP\ VP_1$
$S_2 \rightarrow NP\ VP_2$
$VP_1 \rightarrow V\ NP$
$VP_2 \rightarrow VP_1\ PP$
$VP_2 \rightarrow VP_2\ PP$
$PP \rightarrow P\ NP$
$P \rightarrow$ with
$V \rightarrow$ saw
$NP \rightarrow NP\ PP$
$NP \rightarrow$ I | joy | saw
$NP \rightarrow$ stars | moons

Ex:

- (1,5,S_1) → [(1,1,NP), (2,5,VP_1)]
- (2,5,VP_1) → [(2,2,V), (3,5,NP)]
- ...

Probabilistic parsing

To compute the probability of a parse tree, all we need is to extend the pointers to include the probability of each production.



$S_1 \rightarrow NP\ VP_1$ (0.7)
$S_2 \rightarrow NP\ VP_2$ (0.3)
$VP_1 \rightarrow V\ NP$ (1.0)
$VP_2 \rightarrow VP_1\ PP$ (1.0)
$VP_2 \rightarrow VP_2\ PP$ (1.0)
$PP \rightarrow P\ NP$ (1.0)
$P \rightarrow$ with (1.0)
$V \rightarrow$ saw (1.0)
$NP \rightarrow NP\ PP$ (0.4)
$NP \rightarrow$ I (0.1) | joy (0.1) | saw (0.1)
$NP \rightarrow$ stars (0.2) | moons (0.1)

Ex:

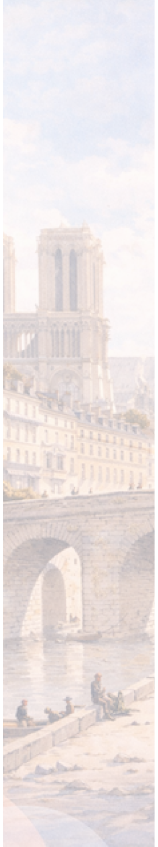
- (1,5,S_1,**0.7**) → [(1,1,NP,**0.1**), (2,5,VP_1,**1**)]
- (2,5,VP_1,**1**) → [(2,2,V,**1**), (3,5,NP,**0.4**)]
- ...

NOTE: The probabilities in red reflect the original probabilities for each "kind" of VP.



Relationship to other modules

1. Semantic parsing
    - Intent classification
    - Slot filling
2. Parts of Speech and POS Tagging
3. Language models and their "grammar" knowledge.



What you should know or be able to do

- Use grammars to generate text and to check if text is grammatical.
- Explain the differences and similarities between constituency and dependency parse trees.
- Identify relations or constituents and their parts within constituent and dependency parse trees.
- Explain how to perform information extraction from constituent and dependency parse trees.
- Manually perform the CFG → CNF conversion and manually execute the CYK algorithm.
- Explain what kinds of language resources helped with the tasks above and how they could be built.